

specs introduce a number of features to make offline web apps a reality like application cache, local storage, and indexedDB.

There are several reasons to use client-side storage. First, you can make your app work when the user is offline, possibly sync'ing data back once the network is connected again. Second, it's a performance booster; you can show a large corpus of data as soon as the user clicks on to your site, instead of waiting for it to download again. Third, it's an easier programming model, with no server infrastructure required. Of course, the data is more vulnerable and the user can't access it from multiple clients, so you should only use it for non-critical data, in particular cached versions of data that's also "in the cloud".

Session and Local Storage for web applications

Web Storage simply provides a key-value mapping, e.g. `localStorage["name"] = username;`. Unfortunately, present implementations only support string-to-string mappings, so you need to serialise and de-serialise other data structures. You can do so using `JSON.stringify()` and `JSON.parse()`.

There are two types of storage, `sessionStorage` and `localStorage`. Session Storage is primarily intended for scenarios where the user is carrying out a single transaction. Information that is put in the store is available across all windows tabs and frames for the entire session the browser stays open. Local Storage persists across sessions, so even after closing and reopening the browser, the data is still available.

Browser support for session and local storage is pretty good. All the new and popular browsers support it. You can use Modernizr to detect support for Web Storage.

```

• if (Modernizr.localstorage) {
•   // window.localStorage is available!
• } else {
•   // no native support for HTML5 storage :(
•   // maybe try dojox.storage or a third-party solution
• }

```

Setting up Web Storage

For `localStorage` or `sessionStorage`, we only need to check the store exists and is of the right type. If not, we'll create a new array and store it against the `localStorage` "checkins" key. We use JSON to convert the structure to a